

Advanced Programming

Assignment Submission | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 16
Semester	Spring 2026
Language	C (GCC + POSIX Threads)

Producer-Consumer System Using Semaphores

This assignment implements a multithreaded Producer-Consumer system in C using POSIX threads and semaphores. Multiple producer threads generate items into a shared circular buffer while consumer threads remove and process them — all coordinated safely using three semaphores.

Question

Develop a multithreaded C program using POSIX threads where multiple threads coordinate access to a shared resource using semaphores. The program should ensure that threads wait correctly when the resource is unavailable and continue execution only when signaled.

The program must demonstrate:

- Thread creation using `pthread_create()`
- Synchronization using `sem_wait()` and `sem_post()`
- Thread completion using `pthread_join()`
- Print messages showing thread execution order
- Explanation of how synchronization prevents inconsistent behaviour

System Design

Buffer and Semaphore Layout

SHARED CIRCULAR BUFFER (size 5)

```
+-----+-----+-----+-----+
| 1   | 2   |   |   |   |
+-----+-----+-----+-----+
      ^
      |__ Producer adds here (in)
          ^
          |__ Consumer takes (out)
```

3 SEMAPHORES:

- empty = 5 (free slots) -- blocks producer when buffer full
- filled = 0 (ready items) -- blocks consumer when buffer empty
- mutex = 1 (binary lock) -- only 1 thread touches buffer at a time

Semaphore Roles

Semaphore	Initial Value	Producer does	Consumer does
empty	BUFFER_SIZE (5)	sem_wait(&empty) before adding	sem_post(&empty) after taking
filled	0	sem_post(&filled) after adding	sem_wait(&filled) before taking
mutex	1 (binary)	sem_wait(&mutex) before writing	sem_wait(&mutex) before reading

Source Code — producer_consumer.c

Save as producer_consumer.c inside Assignment16 folder.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

/* --- Shared buffer ----- */
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

/* --- Semaphores ----- */
sem_t empty; /* free slots - starts at BUFFER_SIZE */
sem_t filled; /* ready items - starts at 0 */
sem_t mutex; /* binary lock - only 1 thread at a time */

/* --- Shared counters ----- */
int produced = 0; /* total items produced so far */
int consumed = 0; /* total items consumed so far */
pthread_mutex_t count_lock = PTHREAD_MUTEX_INITIALIZER;

/* --- Producer ----- */
void *producer(void *arg) {
```

```

    int id = *((int *)arg);

    while (1) {
        /* safely grab next item number */
        pthread_mutex_lock(&count_lock);
        if (produced >= NUM_ITEMS) {
            pthread_mutex_unlock(&count_lock);
            break;
        }
        int item = ++produced;
        pthread_mutex_unlock(&count_lock);

        /* wait for free slot then lock buffer */
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("[Producer %d] Produced item %2d -> buffer[%d]\n", id, item, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&filled); /* signal one item is ready */
    }

    printf("[Producer %d] Done.\n", id);
    return NULL;
}

/* — Consumer ————— */
void *consumer(void *arg) {
    int id = *((int *)arg);

    while (1) {
        /* safely check if all items already consumed */
        pthread_mutex_lock(&count_lock);
        if (consumed >= NUM_ITEMS) {
            pthread_mutex_unlock(&count_lock);
            break;
        }
        consumed++;
        pthread_mutex_unlock(&count_lock);

        /* wait for item then lock buffer */
        sem_wait(&filled);
        sem_wait(&mutex);

        int item = buffer[out];
        printf("[Consumer %d] Consumed item %2d <- buffer[%d]\n", id, item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty); /* signal one slot is free */
    }

    printf("[Consumer %d] Done.\n", id);
    return NULL;
}

/* — Main ————— */
int main(void) {
    pthread_t prod_threads[NUM_PRODUCERS];
    pthread_t cons_threads[NUM_CONSUMERS];
    int prod_ids[NUM_PRODUCERS];
    int cons_ids[NUM_CONSUMERS];

    printf("=====\n");
    printf("    PRODUCER-CONSUMER WITH SEMAPHORES\n");

```

```

printf("=====\n");
printf("Buffer size : %d\n", BUFFER_SIZE);
printf("Total items : %d\n", NUM_ITEMS);
printf("Producers : %d\n", NUM_PRODUCERS);
printf("Consumers : %d\n", NUM_CONSUMERS);
printf("-----\n\n");

/* initialise semaphores */
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&filled, 0, 0);
sem_init(&mutex, 0, 1);

/* create producers */
for (int i = 0; i < NUM_PRODUCERS; i++) {
    prod_ids[i] = i + 1;
    pthread_create(&prod_threads[i], NULL, producer, &prod_ids[i]);
}

/* create consumers */
for (int i = 0; i < NUM_CONSUMERS; i++) {
    cons_ids[i] = i + 1;
    pthread_create(&cons_threads[i], NULL, consumer, &cons_ids[i]);
}

/* wait for all to finish */
for (int i = 0; i < NUM_PRODUCERS; i++)
    pthread_join(prod_threads[i], NULL);
for (int i = 0; i < NUM_CONSUMERS; i++)
    pthread_join(cons_threads[i], NULL);

/* cleanup */
sem_destroy(&empty);
sem_destroy(&filled);
sem_destroy(&mutex);

printf("\n-----\n");
printf("Total produced : %d\n", NUM_ITEMS);
printf("Total consumed : %d\n", consumed);
printf("=====\n");
return 0;
}
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

/* — Shared buffer ————— */
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

/* — Semaphores ————— */
sem_t empty; /* free slots - starts at BUFFER_SIZE */
sem_t filled; /* ready items - starts at 0 */
sem_t mutex; /* binary lock - only 1 thread at a time */

/* — Shared counters ————— */
int produced = 0; /* total items produced so far */
int consumed = 0; /* total items consumed so far */
pthread_mutex_t count_lock = PTHREAD_MUTEX_INITIALIZER;

/* — Producer ————— */

```

```

void *producer(void *arg) {
    int id = *((int *)arg);

    while (1) {
        /* safely grab next item number */
        pthread_mutex_lock(&count_lock);
        if (produced >= NUM_ITEMS) {
            pthread_mutex_unlock(&count_lock);
            break;
        }
    }
}

```

Compile and Run

```

cd "D:\Downloads D Folder\Advanced Programming 2\Assignment16"
gcc producer_consumer.c -o producer_consumer -lpthread
.\producer_consumer

```

Expected Output

Note: exact thread interleaving order may vary each run — that is normal for multithreaded programs.

```

=====
      PRODUCER-CONSUMER WITH SEMAPHORES
=====
Buffer size   : 5
Total items   : 10
Producers     : 2
Consumers     : 2
-----

[Producer 1] Produced item 1 -> buffer[0]
[Producer 2] Produced item 2 -> buffer[1]
[Consumer 1] Consumed item 1 <- buffer[0]
[Consumer 2] Consumed item 2 <- buffer[1]
[Producer 1] Produced item 3 -> buffer[2]
[Producer 2] Produced item 4 -> buffer[3]
[Consumer 1] Consumed item 3 <- buffer[2]
[Producer 1] Produced item 5 -> buffer[4]
[Producer 2] Produced item 6 -> buffer[0]
[Consumer 2] Consumed item 4 <- buffer[3]
[Producer 1] Produced item 7 -> buffer[1]
[Producer 2] Produced item 8 -> buffer[2]
[Consumer 1] Consumed item 5 <- buffer[4]
[Producer 1] Produced item 9 -> buffer[3]
[Producer 2] Produced item 10 -> buffer[4]
[Producer 1] Done -- exiting.
[Producer 2] Done -- exiting.
[Consumer 2] Consumed item 6 <- buffer[0]
[Consumer 1] Consumed item 7 <- buffer[1]
[Consumer 2] Consumed item 8 <- buffer[2]
[Consumer 1] Consumed item 9 <- buffer[3]
[Consumer 2] Consumed item 10 <- buffer[4]
[Consumer 1] Done -- exiting.
[Consumer 2] Done -- exiting.

-----
All items produced and consumed.
Total produced : 10
Total consumed : 10
=====

```

Key Concepts

Function / Concept	Where Used	What it Does
<code>sem_wait(&empty)</code>	<code>Producer</code>	Blocks producer when buffer is full — waits for free slot
<code>sem_post(&empty)</code>	<code>Consumer</code>	Signals producer that a slot is now free after consuming
<code>sem_wait(&filled)</code>	<code>Consumer</code>	Blocks consumer when buffer is empty — waits for item
<code>sem_post(&filled)</code>	<code>Producer</code>	Signals consumer that a new item has been added
<code>sem_wait(&mutex)</code>	<code>Both</code>	Ensures only one thread accesses the buffer at a time
<code>sem_post(&mutex)</code>	<code>Both</code>	Releases buffer access so the next thread can enter
<code>pthread_create()</code>	<code>main()</code>	Spawns producer and consumer threads
<code>pthread_join()</code>	<code>main()</code>	Waits for all threads to complete before program exits
<code>sem_init()</code>	<code>main()</code>	Initialises each semaphore with its starting value
<code>sem_destroy()</code>	<code>main()</code>	Cleans up semaphore resources after all threads finish
Circular buffer	<code>buffer[]</code>	in and out indices wrap around using <code>% BUFFER_SIZE</code>
<code>sleep(1) / sleep(2)</code>	<code>Both threads</code>	Simulates real work time — makes thread interleaving visible

Summary

- We created a shared circular buffer of size 5 where producer threads add items and consumer threads remove them, coordinated entirely through semaphores.
- Three semaphores control everything — `empty` tracks free slots and blocks the producer when full, `filled` tracks available items and blocks the consumer when empty, and `mutex` ensures only one thread touches the buffer at a time.
- `sem_wait()` makes a thread sleep and wait when the resource is unavailable, and `sem_post()` wakes up the waiting thread when the resource becomes available — this is how threads communicate without clashing.
- Without semaphores both threads would read and write the buffer simultaneously causing corrupted data, but with semaphores every produce and consume operation happens in the correct order with zero data loss.

Output Screenshots

Terminal — full program output showing producer and consumer coordination:

Compiled Successfully. memory: 3840 time: 0.01 exit code: 0

```
=====
      PRODUCER-CONSUMER WITH SEMAPHORES
=====
Buffer size   : 5
Total items   : 10
Producers     : 2
Consumers     : 2
-----

[Producer 2] Produced item 1 -> buffer[0]
[Producer 2] Produced item 2 -> buffer[1]
[Producer 2] Produced item 3 -> buffer[2]
[Producer 2] Produced item 4 -> buffer[3]
[Producer 2] Produced item 5 -> buffer[4]
[Consumer 1] Consumed item 1 <- buffer[0]
[Producer 1] Produced item 7 -> buffer[0]
[Consumer 1] Consumed item 2 <- buffer[1]
[Consumer 1] Consumed item 3 <- buffer[2]
[Consumer 1] Consumed item 4 <- buffer[3]
[Consumer 1] Consumed item 5 <- buffer[4]
[Consumer 1] Consumed item 7 <- buffer[0]
[Producer 2] Produced item 6 -> buffer[1]
[Producer 2] Produced item 9 -> buffer[2]
[Producer 2] Produced item 10 -> buffer[3]
[Producer 2] Done.
[Consumer 1] Consumed item 6 <- buffer[1]
[Consumer 1] Consumed item 9 <- buffer[2]
[Consumer 1] Consumed item 10 <- buffer[3]
[Producer 1] Produced item 8 -> buffer[4]
[Producer 1] Done.
[Consumer 1] Consumed item 8 <- buffer[4]
[Consumer 1] Done.
[Consumer 2] Done.

-----
Total produced : 10
Total consumed : 10
=====
```